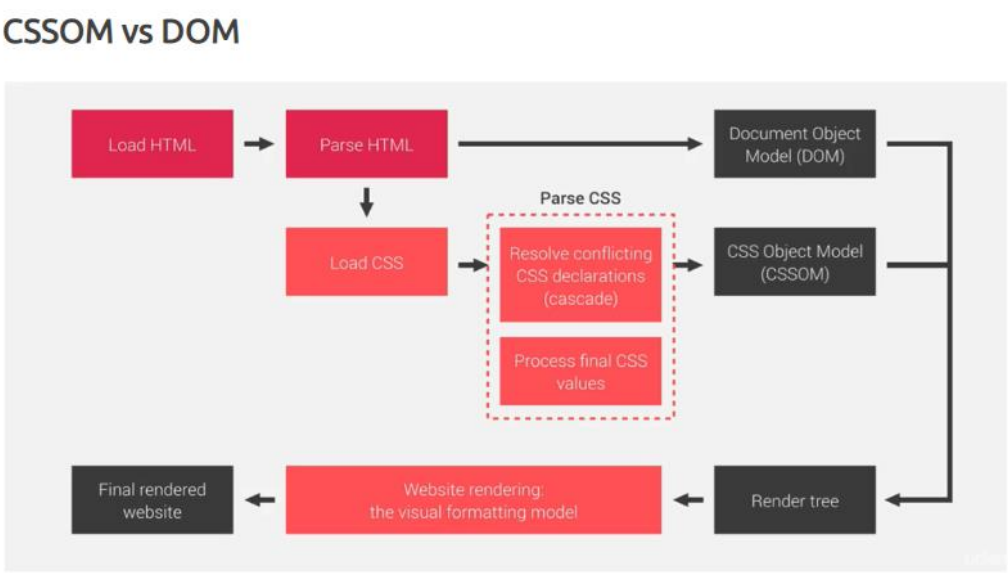
01



02

```
> typeof address;
< 'string'
> var address = "galle";
< undefined
> typeof address;
< 'string'
> address = 10;
< 10
> typeof address;
< 'number'
```

03

```
> var name = "amal";
  var name = "kamal";
  console.log(name);
  kamal
```

04

```
{
    var z = 10;
}

console.log(z);
10
```

05

```
> console.log(y);
  var y=10;
  console.log(y);
  undefined
  10
```

06

```
names.forEach(function (v){
  console.log(v);
});
```

07

```
for(var i in names){
  console.log( i,names[i]);// i refer index of the array
}
```

08

```
for(var i of names){
  console.log(i);// i refer values of the array
}
```

09

```
function greet() {
   console.log("Hello, World!");
}
```

10

```
function greetUser(name) {  // 'name' is a parameter
   console.log("Hello, " + name + "!");
}
greetUser("Alice");  // 'Alice' is an argument passed to the function
// Output: Hello, Alice!
```

11

```
const sayHello = function() {  // Function expression
   console.log("Hello!");
};
sayHello();  // Output: Hello!
```

12

```
const greet = (name) => {
   return "Hello, " + name;
};
console.log(greet("John"));  // Output: Hello, John
```

13

```
const square = num => num * num;
console.log(square(4));  // Output: 16
```

14

```
const showMessage = function() {
    console.log("This is an anonymous function!");
};
showMessage();  // Output: This is an anonymous function!
```

15

```
function executeFunction(fn) {
    fn();  // Calls the function passed as an argument
}
executeFunction(function() {
    console.log("Hello from a higher-order function!");
});
```

16

```
(function() {
    console.log("This is an IIFE!");
})();
```

17

```
function outer() {
    let outerVar = "I am from outer function!";

    function inner() {
        console.log(outerVar);  // inner function can access outerVar
    }
    return inner;
}
const innerFunction = outer();
innerFunction();  // Output: I am from outer function!
```

18

```
function greet(name = "Guest") {
    console.log("Hello, " + name);
}
greet();  // Output: Hello, Guest
greet("Alice");  // Output: Hello, Alice
```

19

```
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}
console.log(sum(1, 2, 3, 4));  // Output: 10
```

20

```
const arr = [1, 2, 3];
console.log(...arr);  // Output: 1 2 3
```

21

Html

```
<p id="message">Hello, World!</p>
```

js

```
const message = document.getElementById("message");
// Change the content
message.innerHTML = "Hello, DOM Manipulation!";
message.textContent = "Updated Text Content";  // This will only set plain text
```

22

html

```
<img id="myImage" src="image1.jpg">
```

js

```
const image = document.getElementById("myImage");
// Changing attributes
image.setAttribute("src", "image2.jpg");
// OR
image.src = "image2.jpg";
```

23

html

```
<div id="box" style="width: 100px; height: 100px; background-color: red;"></div>
```

js

```
const box = document.getElementById("box");
// Change style
box.style.backgroundColor = "blue";
box.style.width = "200px";
```

24

```
const elementToRemove = document.getElementById("box");
elementToRemove.remove();  // Removes the element
```

25

Html

```
<button id="myButton">Click Me!</button>
```

js

```
const button = document.getElementById("myButton");
button.addEventListener("click", function() {
    alert("Button was clicked!");
});
```

26

```
const parent = document.getElementById("box").parentNode;  // Get parent element
const children = document.getElementById("box").children;  // Get child elements
const nextSibling = document.getElementById("box").nextSibling;  // Get next sibling
const previousSibling = document.getElementById("box").previousSibling;  // Get previous sibling
```

27

```
fruits.push("Grapes");  // Adds "Grapes"
console.log(fruits);  // ["Apple", "Mango", "Orange", "Grapes"]

fruits.pop();  // Removes "Grapes"
console.log(fruits);  // ["Apple", "Mango", "Orange"]
```

28

```
fruits.unshift("Strawberry");  // Adds "Strawberry" at the beginning
console.log(fruits);  // ["Strawberry", "Apple", "Mango", "Orange"]

fruits.shift();  // Removes "Strawberry"
console.log(fruits);  // ["Apple", "Mango", "Orange"]
```

29

```
// Remove 1 element at index 1 ("Mango") and insert "Pineapple"
fruits.splice(1, 1, "Pineapple");
console.log(fruits);  // ["Apple", "Pineapple", "Orange"]
```

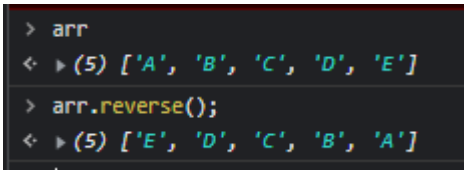
30

```
const slicedFruits = fruits.slice(0, 2);  // Copy first two elements
console.log(slicedFruits);  // ["Apple", "Pineapple"]
```

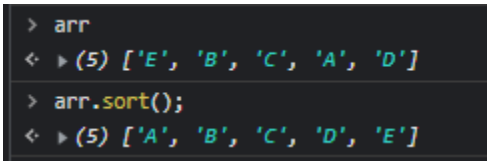
31

```
const index = fruits.indexOf("Orange");
console.log(index);  // 2
```

32



33



35

```
fruits.forEach(function(fruit) {
    console.log(fruit);
});
// Same Output
```

34

```
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
// Output:
// Apple
// Pineapple
// Orange
```

36

```
for (let fruit of fruits) {
    console.log(fruit);
}
// Same Output
```

37

```
setTimeout(function () {
   console.log("Hello there how are you.?")
}, 2000);
```

38

```
var testTimer = setTimeout(function () {
   console.log("Hello there how are you.?")
}, 2000);
// stop timeout
clearTimeout(testTimer);
```

39

```
let x = 0;
setInterval(function () {
   x++;
   console.log(x);
}, 1000);
```

40

```
let x = 0;
var timerId = setInterval(function () {
  x++;
  console.log(x);
  if (x > 5) {
     clearInterval(timerId);// stop the set Interval
  }
}, 1000);
```

41

```
let decimal=20;
console.log(typeof decimal);//number

let binaryNumber=0b101;
console.log(typeof binaryNumber);//number

let octalNumber=0o17;
console.log(typeof octalNumber);//number

let hexaDecimal=0xA;
console.log(typeof hexaDecimal);//number

let floatingPoint=20.34;
console.log(typeof floatingPoint);//number
```

String related methods

```
let sampleText = " Hello, Hi there! ";

console.log(sampleText); //  Hello, Hi there!


let a = sampleText.toUpperCase();

console.log(a); //  HELLO, HI THERE!


let b = sampleText.toLowerCase();

console.log(b); //  hello, hi there!


let c = sampleText.trim();

//දෙපැත්තෙම white spaces අයින් කරනෝ

console.log(c); //Hello, Hi there!


let d = sampleText.trimRight();

//white spaces අයින් කරනෝ (Right side)

console.log(d); // Hello, Hi there!


let e = sampleText.trimLeft();

//white spaces අයින් කරනෝ (Left side)

console.log(e); //Hello, Hi there! !


let f = sampleText.charAt(5);

//pass කරන index එකට අදාල character එක return කරනෝ

console.log(f); //o


let g = sampleText.charCodeAt(5);

//pass කරන index එකට අදාල character ASCII code එක return කරනෝ

console.log(g); //111 (o වල ASCII code එක)


let h = sampleText.indexOf("e");

//pass කරන character එක මුලින්ම හම්බුවෙන index number එක return කරනෝ

console.log(h); //2


let hh = sampleText.indexOf("A");

//string එකෙ නැති character එකක් නම් -1 return කරනෝ

console.log(hh); //-1


let i = sampleText.substring(0,5);

//දෙන index range එකක් අතර තියෙන string කොටස return කරනෝ

console.log(i); // Hell
```

42

literal base object

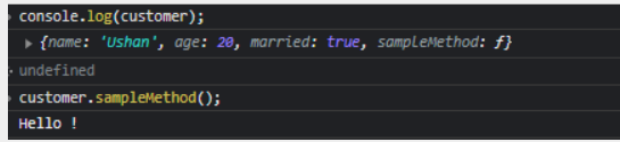
```
let customer = {
   name:"Ushan",
   age :20,
   married: true
}

//access properties
customer.age; // dot notation
customer['age']; // bracket notation
```

43

44

```
customer.sampleMethod = function (){
   console.log("Hello !")
}
```



45

```
function greet(name) {
  console.log("Hello, " + name + "!");
}

greet("John"); // Output: Hello, John!
greet("Alice"); // Output: Hello, Alice!
```

46

```
const add = function (a, b) {
  return a + b;
};

const result = add(3, 4);
console.log(result); // Output: 7
```

47

```
// Traditional function
const add = function(a, b) {
    return a + b;
};

// Arrow function
const addArrow = (a, b) => a + b;

console.log(addArrow(2, 3));  // Output: 5
```

48

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(num => num * 2);
console.log(doubled);  // Output: [2, 4, 6, 8, 10]
```

49

```
function testMethod(id){
   console.log(`Method 1 invoked  ${id}`);
}

function testMethod(id,name){
   console.log(`Method 2 invoked ${id} , ${name}`);
}

// call methods
testMethod(10);
testMethod(10,"John");
```

output ⇒

```
Method 2 invoked 10 , undefined
Method 2 invoked 10 , John
```

(අන්තිමට තියෙන method එක call වෙන්නෙ)

50

```
function testMethod(){
   console.log(arguments);
   //this array stores all the information about method parameters
}

testMethod();
testMethod("test");
testMethod("test",123,false);
```

output ⇒

```
▶ Arguments [callee: ƒ, Symbol(Symbol.iterator): ƒ]
▶ Arguments ['test', callee: ƒ, Symbol(Symbol.iterator): ƒ]
▶ Arguments(3) ['test', 123, false, callee: ƒ, Symbol(Symbol.iterator): ƒ]
```

51

```
<button id="myButton">Click Me</button>

<script>
    const button = document.getElementById('myButton');
    button.addEventListener('click', function() {
        alert('Button was clicked!');
    });
</script>
```

52

```
<button onClick="alert('Clicked')"> Click </button>
Or invoke a method
<button onclick="sampleMethod()"> Click </button>
```

53

```
<button id="btn">Click</button>

let e = document.getElementById('btn');

e.addEventListener('click',function (){
   alert('button clicked !')
});
```

54

```
function greet(name = 'Guest') {
    return `Hello, ${name}!`;
}

console.log(greet()); // Output: Hello, Guest!
console.log(greet('Alice')); // Output: Hello, Alice!
```

55

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // Output: [1, 2, 3, 4, 5]
```

56

```
function sum(...numbers) {
    return numbers.reduce((acc, num) => acc + num, 0);
}

console.log(sum(1, 2, 3)); // Output: 6
```

57